4.8 Model fields

Django 5 ships with 29 data fields and 3 relationship fields. Some fields map directly to an SQL data type, while others are a convenient abstraction adding features on top of the SQL data type. For example, the `EmailField` is usually implemented as text in the underlying database, but it also provides validation to ensure the text input meets the standard format of an email address.

When you define a field, you can pass in arguments that control the construction of the underlying column. You saw this with `max_length` on a `CharField`. Some arguments can be used with all fields, and some are field specific. This section highlights frequently used fields and their arguments. For a complete listing, see the Django Model field reference document: http://mng.bz/nged.

4.8.1 Popular cross-field field options

In the section on one-to-many relationships, you learned abou the `null` and `blank` field options. Both of the options are available across all fields.

The `null` option specifies whether the column in the database can contain the `NULL` value. This is a Boolean option and defaults to `False`. `NULL` in SQL indicates a lack of a value. Depending on the column, this can get a little confusing. A `DateField` configured with `null=True` has two types of state: empty and containing a date value. By contrast, a `CharField` with `null=True` has three types of state: empty (`NULL`), empty string (`''`), and containing a string. Generally it is advised to use an empty string with text and leave `null=False`.

Closely related to `null` is the `blank` option. The `null` option specifies the behavior in the database, whereas the `blank` option specifies the behavior in the Django Admin and web forms. A value of `blank=False` (the default) means the Django Admin and web-based forms treat this field as required. It is possible to have `blank=True` and `null=False` if you are programmatically populating a field between form submission and writing the value to the database. This combination is unusual though—typically the values of these two options are the same.

The separation between `null` and `blank` exposes a design feature of Django: the differentiation between database and model validation. The fields in your `Model` may validate their data. `Model`-level validation happens before values get saved to the database. This is done both to help ensure the content can be saved to the database and to add additional validation for some fields. The `EmailField` example mentioned earlier does this. `Model`-level validation ensures the content meets the format for an email address, whereas the database validation may only care if the content is text. How this is implemented may vary between databases. If your database provides an email data type, Django generally will take advantage of it. If it does not, it will use model validation and then store it as text.

If you are frequently setting a field to the same value, the `default` option can be useful. When set, you no longer need to provide a value for this field when constructing a model. If you do not provide a value, the given default is used instead.